МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Иванов Роман Евгеньевич
(подпись) (фамилия, имя, отчество)

Группы _____ИПо-41_____

Руководитель практической подготовки от профильной организации

_____Декан математического факультета _____
(должность)

_____ Скорнякова Анна Юрьевна
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Оглавление

ВВЕДЕНИЕ 3
РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 5
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ 7
2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении. 7
2.2. Ознакомление с должностными и функциональными обязанностями 7
2.3. Организация информационного обеспечения и защиты информации предприятия 8
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 10
Тема 3.1. Выполнение производственных заданий 10
3.1.1. использование выбранной системы контроля версий; 10
3.1.2. разработка программного обеспечения; 10
3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества; 16
3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования; 18
3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств; 19
3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; 19
3.1.7. Интегрирование программных модулей; 21
3.1.8. Верификация и аттестация программного обеспечения; 22
ЗАКЛЮЧЕНИЕ 24
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 25

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения интеграция модулей в программное обеспечение; отладка программного модуля с использованием специализированных программных средств; разработка тестовых наборов и тестовых сценариев для программного обеспечения.

Базой практики выступил математический факультет ФГБОУ ВО «Пермский государственный гуманитарно-педагогический университет» (ПГГПУ).

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

7. Планировать и реализовывать собственное профессиональное и личностное развитие;

8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

10. Использовать информационные технологии в профессиональной деятельности

11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В ходе организационно-подготовительного этапа мною был пройден вводный и первичный инструктаж по охране труда и технике безопасности, что подтверждается соответствующими записями в журнале регистрации инструктажа. Я ознакомился с правилами внутреннего трудового распорядка ПГГПУ, установленными для сотрудников и обучающихся.

Мною были изучены требования безопасности при работе с ИТ-оборудованием и электрическими приборами, включая правила включения/выключения техники, заземления и недопустимости самостоятельного вскрытия оборудования.

Описание рабочего места, используемой техники и ПО:

Мое рабочее место расположено в помещении кафедры (или в компьютерном классе) математического факультета ПГГПУ. Рабочее место оснащено стандартным набором оргтехники и мебелью: компьютерный стол, стул, системный блок, монитор, клавиатура и мышь.

Техническое оснащение:

- Системный блок:
  - Процессор Intel Core i3/i5
  - оперативная память 8 ГБ
  - жесткий диск объемом 500 ГБ
- Монитор:
  - ЖК-монитор диагональю 21-24 дюйма.
- Оргтехника:
  - На кафедре установлено сетевое МФУ (принтер+сканер+копир) для печати документов.

Программное обеспечение:

- Операционная система: Windows 10 Pro.
- Офисный пакет: Microsoft Office 2016/2019 (Word, Excel, PowerPoint). Используется для подготовки документов, ведения ведомостей и оформления отчетов.
- Интернет и связь: Браузеры Google Chrome или Yandex Browser для выхода в интернет и работы с электронной информационно-образовательной средой (ЭИОС) университета.
- Антивирусная защита: Kaspersky Endpoint Security – обеспечивает безопасность при работе в сети и с внешними носителями.

Специализированное ПО:

Для выполнения заданий практики на компьютере также установлены: редактор кода (Visual Studio), СУБД Postgres и необходимое для разработки ПО.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении.

Структура предприятия и ИТ-подразделения:

Общее руководство университетом осуществляет ректор. Математический факультет возглавляет декан. В состав факультета входят деканат, кафедры и лаборатории. Ключевым подразделением, обеспечивающим учебную, методическую и научную деятельность, является кафедра. На кафедре работает профессорско-преподавательский состав (ППС) и учебно-вспомогательный персонал (УВП), к которому относится лаборант.

Функции ИТ-поддержки и сопровождения на факультете выполняются несколькими уровнями:

Общеуниверситетский уровень: Центр информационных технологий (или аналогичное подразделение) обеспечивает работу серверов, общеуниверситетской сети, ЭИОС и базового ПО.

Уровень факультета/кафедры: Лаборанты и специалисты кафедр, такие как я, отвечают за поддержку оборудования и ПО на своих рабочих местах, в компьютерных классах, помогают преподавателям и студентам в решении текущих технических вопросов. Взаимодействие между подразделениями строится на основе служебных записок и устных заявок.

Характеристика предприятия:

ПГГПУ – это ведущий педагогический вуз Пермского края, осуществляющий подготовку кадров по широкому спектру направлений, включая педагогическое образование в области математики, информатики и экономики. Университет имеет развитую инфраструктуру, включающую учебные корпуса, библиотеку, компьютерные классы и лаборатории.

2.2. Ознакомление с должностными и функциональными обязанностями

В рамках данного этапа мною была изучена должностная инструкция инженера-программиста кафедры информатики и вычислительной техники. Основные обязанности, соответствующие профессиональному модулю ПМ.02:

№	Должностная обязанность	
1	Разработка архитектуры программного обеспечения	Разработка требований к программным модулям

№	Должностная обязанность	
2	Интеграция отдельных модулей в единую систему	Интеграция модулей
3	Отладка программного кода с использованием специализированных средств	Отладка
4	Разработка тестовых сценариев и проведение тестирования	Разработка тестовых наборов
5	Проверка соответствия кода стандартам кодирования	Инспектирование
6	Ведение документации на разрабатываемое ПО	Обеспечивает качество разработки
7	Работа с системой контроля версий Git	Обеспечивает интеграцию модулей

С руководителем практики было согласовано индивидуальное задание, целью которого является разработка и интеграция программных модулей для автоматизации учета партнеров компании.

2.3. Организация информационного обеспечения и защиты информации предприятия.

На математическом факультете ПГГПУ для управления разработкой программного обеспечения используются следующие инструменты:

Системы контроля версий:

- Git — распределенная система управления версиями
- GitHub / GitLab — хостинг удаленных репозиториев для командной работы

Среды разработки:

- Visual Studio 2022 — для разработки на C# и .NET
- Visual Studio Code — для веб-разработки
- PyCharm — для разработки на Python

Системы управления базами данных:

- PostgreSQL — основная СУБД для разрабатываемых приложений
- MySQL — используется в учебном процессе
- SQL Server — для корпоративных решений

Инструменты тестирования:

- MSTest / NUnit — для модульного тестирования
- Postman — для тестирования API
- Selenium — для автоматизированного тестирования интерфейсов

Средства документирования:

- Doxygen / Sandcastle — генерация документации из комментариев к коду
- XML-комментарии в C# — документирование API

Интеграция информационных систем:

Разрабатываемое приложение "Учет партнеров" интегрируется с:

- ЭИОС Moodle (для авторизации пользователей)
- 1С: Бухгалтерия (для обмена данными о продажах)
- Корпоративной почтой (для уведомлений)

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Выполнение производственных заданий

3.1.1. использование выбранной системы контроля версий;

В процессе разработки программного продукта "Учет партнеров" мною была выбрана система контроля версий Git. Git является распределенной системой управления версиями, которая позволяет отслеживать изменения в исходном коде, работать над проектом в команде и сохранять историю разработки.

Основные принципы работы Git:

- Локальный репозиторий - хранит всю историю изменений на локальном компьютере
- Удаленный репозиторий - размещен на сервере (GitHub) для синхронизации и резервного копирования
- Коммиты - фиксация изменений с описанием выполненных задач
- Ветки (branches) - параллельная разработка различных функций

В ходе выполнения индивидуального задания был создан репозиторий PartnerAccounting_Ivanov куда был помещен весь проект, так же сделал его публичным, чтобы была возможность проверить его.

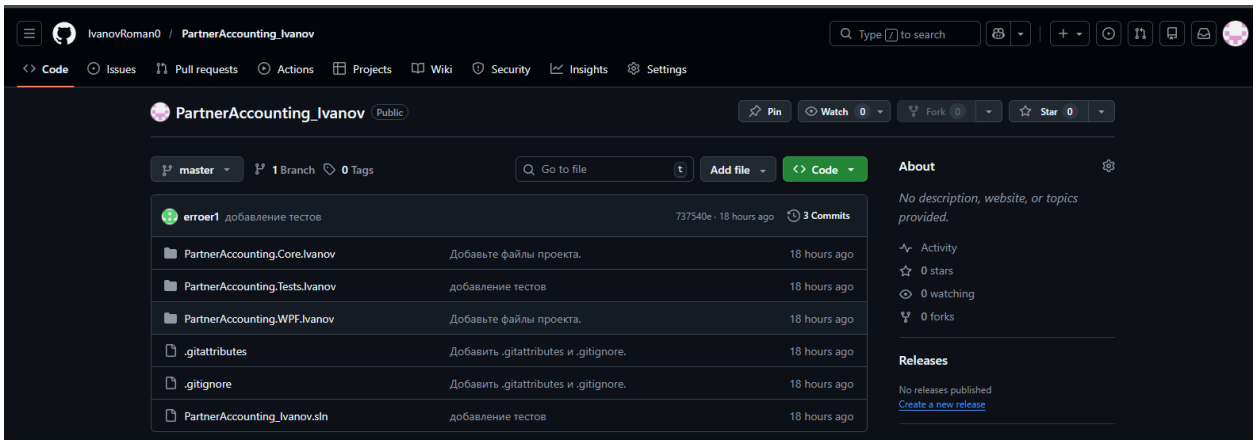


Рисунок 1 (репозиторий Git)

3.1.2. разработка программного обеспечения;

Выбор модели разработки

Для разработки программного продукта "Учет партнеров" была выбрана **итерационная модель разработки**. Обоснование выбора:

Критерий	Обоснование
Четкие требования	Требования были определены в задании, но

	уточнялись в процессе
Возможность демонстрации	Каждая итерация давала рабочий продукт для демонстрации руководителю
Гибкость	Возможность вносить изменения на основе обратной связи
Поэтапная реализация	Функционал добавлялся последовательно: БД → модели → сервисы → интерфейс

Итерации разработки:

Итерация 1: Проектирование базы данных и создание моделей

Итерация 2: Разработка сервисов для работы с данными

Итерация 3: Создание пользовательского интерфейса

Итерация 4: Реализация функционала истории продаж

Итерация 5: Тестирование и отладка

Инструменты для создания продукта

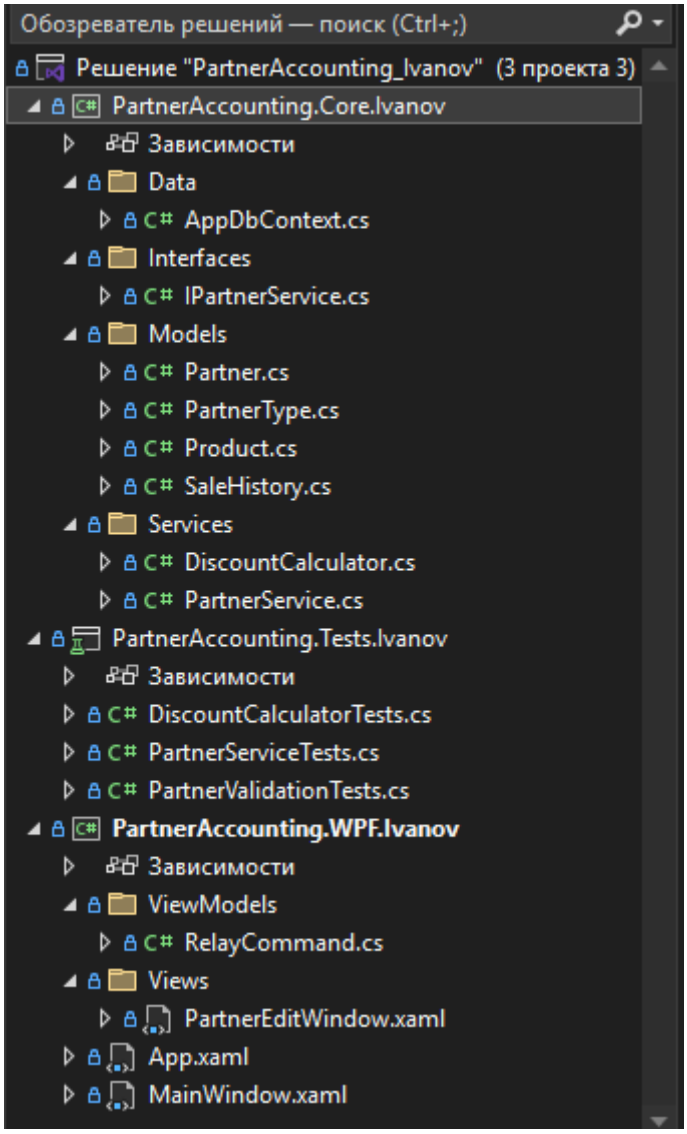
Инструмент	Назначение
Visual Studio 2022	Среда разработки
PostgreSQL	Система управления базами данных
Git	Система контроля версий
GitHub	Хостинг удаленного репозитория
pgAdmin	Визуальное управление БД
Npgsql	Провайдер PostgreSQL для .NET
Entity Framework Core	ORM для работы с БД
MSTest	Фреймворк для модульного тестирования

Среда программирования

В качестве среды программирования использовалась **Microsoft Visual Studio 2022**.

Основные характеристики:

- **Язык программирования:** C# (версия 10.0)
- **Платформа:** .NET 8.0
- **Технология интерфейса:** WPF (Windows Presentation Foundation)
- **Стиль кодирования:** CamelCase для идентификаторов



Рисунко 2 (Интерфейс Visual Studio с открытым проектом)

Диаграмма классов

В процессе разработки была создана диаграмма классов, отражающая структуру программного продукта:

Рисунок 3 (Диаграмма классов)

Этапы разработки ПО

1. Выбор шаблона

Был выбран шаблон WPF Application (.NET) с последующим разделением на три проекта:

- **Core** - бизнес-логика и модели данных
- **WPF** - пользовательский интерфейс
- **Tests** - модульные тесты

2. Составление схемы информационной базы

Проектирование базы данных в 3 нормальной форме:

```
CREATE TABLE app.partner_types (
   id SERIAL PRIMARY KEY,
   name VARCHAR(50) NOT NULL UNIQUE
);

-- Таблица партнеров
```

```sql
CREATE TABLE app.partners (
   id SERIAL PRIMARY KEY,
   type_id INTEGER NOT NULL REFERENCES app.partner_types(id),
   name VARCHAR(200) NOT NULL,
   legal_address VARCHAR(300) NOT NULL,
   inn VARCHAR(12) NOT NULL UNIQUE,
   director_name VARCHAR(200) NOT NULL,
   phone VARCHAR(20) NOT NULL,
   email VARCHAR(100) NOT NULL,
   rating INTEGER NOT NULL DEFAULT 0,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица продукции
CREATE TABLE app.products (
   id SERIAL PRIMARY KEY,
   article VARCHAR(50) NOT NULL UNIQUE,
   name VARCHAR(200) NOT NULL,
   description TEXT,
   min_partner_price DECIMAL(10,2) NOT NULL
);

-- Таблица истории продаж
CREATE TABLE app.sales_history (
   id SERIAL PRIMARY KEY,
   partner_id INTEGER NOT NULL REFERENCES app.partners(id) ON DELETE CASCADE,
   product_id INTEGER NOT NULL REFERENCES app.products(id),
   quantity INTEGER NOT NULL CHECK (quantity > 0),
   sale_date DATE NOT NULL DEFAULT CURRENT_DATE,
   total_amount DECIMAL(10,2) NOT NULL
);
```

3. Тестирование

Разработаны модульные тесты для проверки функциональности:

- Тестирование расчета скидок для всех граничных значений
- Тестирование валидации данных
- Тестирование сервисов

Разработка интерфейса

Главный интерфейс был спроектирован в соответствии с заданием и включает:

- Меню программы (Офис, Партнеры, Справка)
- Список партнеров в виде карточек
- Панель с историей заказов выбранного партнера

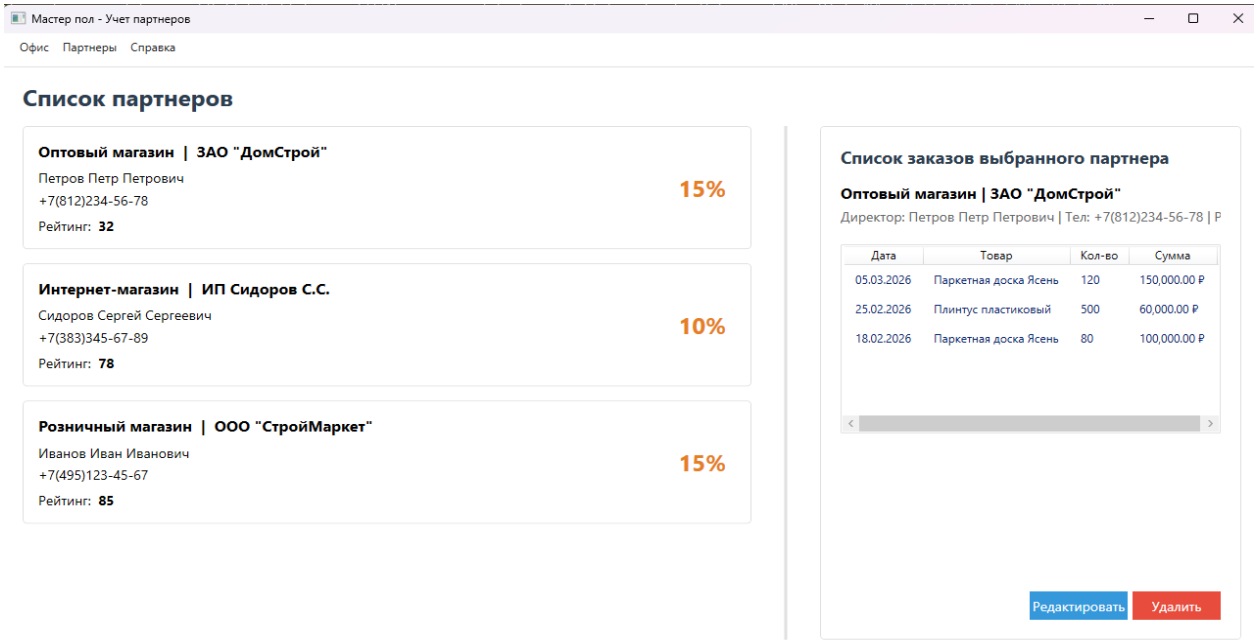


Рисунок 4 (главный интерфейс приложения)

Интерфейс добавления и редактирования партнера включает в себя:

1. Выбор типа партнера
2. Наименование компании
3. Юридический адрес
4. ИНН
5. ФИО директора
6. Телефон
7. Email
8. Рейтинг

Все поля обязательны к заполнению!

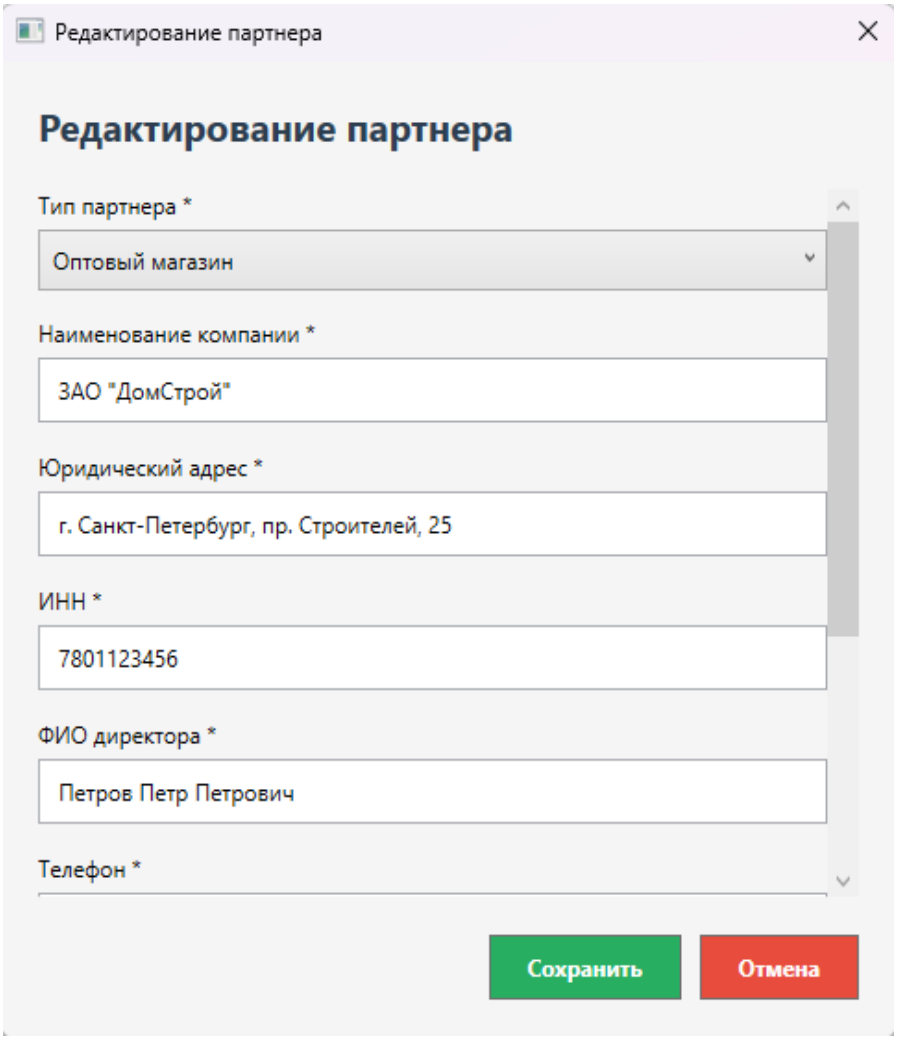


Рисунок 5 (интерфейс добавления\редактирования партнера)

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества;

В процессе разработки программы **«Учет партнеров»** применялись следующие методы и подходы, которые позволили получить качественный и работающий код:

1. Объектно-ориентированное программирование (ООП)

Весь код построен на принципах ООП, что обеспечивает:

Разделение ответственности – каждый класс отвечает за свою задачу:

- Partner – данные о партнере
- PartnerService – логика работы с партнерами
- MainWindow – пользовательский интерфейс

Повторное использование кода – созданные классы можно использовать в разных частях программы

Класс Partner инкапсулирует данные о партнере

```
public class Partner
{
```

```
    public int Id { get; set; }
    public string Name { get; set; }
    public string Phone { get; set; }

    // Метод для расчета скидки
    public int CalculateDiscount(decimal totalSales)
    {
        if (totalSales < 10000) return 0;
        if (totalSales < 50000) return 5;
        if (totalSales < 300000) return 10;
        return 15;
    }
}
```

2. Разделение на слои (трехуровневая архитектура)

Приложение разделено на три логических слоя:

Слой	Назначение	Классы
Слой данных	Работа с базой данных	AppDbContext
Слой бизнес-логики	Расчеты и правила	PartnerService, DiscountCalculator
Слой представления	Интерфейс пользователя	MainWindow, PartnerEditWindow

Такое разделение позволяет:

- Легко находить и исправлять ошибки
- Изменять один слой без изменения других
- Повторно использовать код

3. Обработка исключений

Во всех местах, где могут возникнуть ошибки, добавлена обработка исключений с понятными сообщениями для пользователя:

```
private void AddPartner_Click(object sender, RoutedEventArgs e)
 {
    try
    {
        var window = new PartnerEditWindow();
        window.Owner = this;
        window.WindowStartupLocation = WindowStartupLocation.CenterOwner;

        if (window.ShowDialog() == true)
        {
            LoadPartners();
        }
    }
    catch (Exception ex)
```

```
    {
        MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка",
            MessageBoxButton.OK, MessageBoxImage.Error);
    }
}
```

4. Валидация входных данных

Перед сохранением данных выполняется их проверка:

```
if (!Regex.IsMatch(inn, @"^\d{10}$|^\d{12}$"))
{
    ShowWarning("ИНН должен содержать 10 или 12 цифр");
    InnBox.Focus();
    return false;
}

if (!int.TryParse(RatingBox.Text, out int rating) || rating < 0)
{
    ShowWarning("Рейтинг должен быть целым положительным числом");
    RatingBox.Focus();
    return false;
}
```

5. Тестирование ключевых функций

Для проверки правильности работы созданы модульные тесты:

```
[TestMethod]
public void CalculateDiscount_SalesLessThan10000_Returns0()
{
    // Arrange
    decimal sales = 5000;

    // Act
    int discount = _partner.CalculateDiscount(sales);

    // Assert
    Assert.AreEqual(0, discount, "При сумме продаж менее 10 000 руб. скидка должна быть 0%");
}
```

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

Виды инспекции:

- **Ручная инспекция** - проверка кода на соответствие стандартам
- **Автоматическая инспекция** - с использованием инструментов

Актуальные стандарты кодирования (C# Coding Conventions):

1. **Именование:**

   - PascalCase для классов и методов

   - camelCase для локальных переменных и параметров

   - _camelCase для приватных полей

2. **Форматирование:**

   - Отступы - 4 пробела

   - Фигурные скобки на новой строке

   - Пустые строки для группировки кода

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;

Отладка — это процесс поиска и исправления ошибок в программе. Она нужна, чтобы убедиться, что приложение работает правильно и не падает.

Принцип действия отладки

1. **Запуск программы в режиме отладки** (F5 в Visual Studio)
2. **Установка точек останова** в подозрительных местах
3. **Пошаговое выполнение кода** для отслеживания значений переменных
4. **Анализ результатов** и исправление ошибок

Какие проблемы были решены с помощью отладки

Проблема 1: Ошибка с датой

Ситуация: При сохранении партнера программа падала с ошибкой.

Проблема 2: Ошибка со скидкой

Ситуация: При сумме продаж 300 000 руб. скидка показывала 0% вместо 15%.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения;

Процесс тестирования:

1. **Модульное тестирование (Unit testing)**
   - Тестирование отдельных компонентов

- Использование фреймворка MSTest

2. **Разработка тестовых сценариев**

Тест-кейсы для расчета скидок:

№	Входные данные	Ожидаемый результат	Проверяемая граница
1	5 000 руб.	0%	Менее 10 000
2	10 000 руб.	5%	Равно 10 000
3	25 000 руб.	5%	Между 10 000 и 50 000
4	50 000 руб.	10%	Равно 50 000
5	150 000 руб.	10%	Между 50 000 и 300 000
6	300 000 руб.	15%	Равно 300 000
7	500 000 руб.	15%	Более 300 000
8	0 руб.	0%	Нулевая сумма

```
[TestMethod]
 public void CalculateDiscount_SalesBetween10000And50000_Returns5()
 {
    // Arrange
    decimal sales = 25000;

    // Act
    int discount = _partner.CalculateDiscount(sales);

    // Assert
    Assert.AreEqual(5, discount, "При сумме продаж от 10 000 до 50 000 руб. скидка должна быть
5%");
 }

 [TestMethod]
 public void CalculateDiscount_SalesExactly50000_Returns10()
 {
    // Arrange
    decimal sales = 50000;

    // Act
    int discount = _partner.CalculateDiscount(sales);

    // Assert
    Assert.AreEqual(10, discount, "При сумме продаж 50 000 руб. скидка должна быть 10%");
 }
```

Результаты тестирования:

Всего было разработано **23 unit-теста**:

- DiscountCalculatorTests - 12 тестов (проверка всех граничных значений скидок)
- PartnerValidationTests - 7 тестов (валидация данных)
- PartnerServiceTests - 4 теста (логика сервисов)

Тестирование	Длительн...	При
PartnerAccounting.Tests.Ivanov (23)	1,8 с	
PartnerAccounting.Tests.Ivanov (23)	1,8 с	
DiscountCalculatorTests (12)	88 мс	
CalculateDiscount_SalesBetwee...	86 мс	
CalculateDiscount_SalesBetwee...	< 1 мс	
CalculateDiscount_SalesExactly...	< 1 мс	
CalculateDiscount_SalesExactly...	< 1 мс	
CalculateDiscount_SalesExactly...	< 1 мс	
CalculateDiscount_SalesLessTha...	< 1 мс	
CalculateDiscount_SalesMoreTh...	< 1 мс	
CalculateDiscount_SalesNegativ...	< 1 мс	
CalculateDiscount_SalesZero_R...	< 1 мс	
DiscountDisplay_WithNullSales...	< 1 мс	
DiscountDisplay_WithoutSalesH...	2 мс	
DiscountDisplay_WithSalesHist...	< 1 мс	
PartnerServiceTests (4)	1,7 с	
CalculateDiscount_WithVarious...	6 мс	
GetPartnerDiscount_WithSales...	1,7 с	
PartnerTypeAssignment_ValidTy...	< 1 мс	
ValidatePartnerInn_InnLength10...	< 1 мс	
PartnerValidationTests (7)	7 мс	
ValidatePartner_EmailInvalid_Re...	< 1 мс	
ValidatePartner_InnIs10Digits_V...	< 1 мс	
ValidatePartner_InnIs12Digits_V...	< 1 мс	
ValidatePartner_NameIsEmpty_...	< 1 мс	
ValidatePartner_PhoneNotEmpt...	< 1 мс	
ValidatePartner_RatingIsNegati...	6 мс	
ValidatePartner_ValidData_Pass...	1 мс	

Рсиунок 6 (результаты тестирования)

3.1.7. Интегрирование программных модулей;

Интеграция модулей — это соединение отдельных частей программы в единую работающую систему.

Как соединялись модули в моем приложении

Мое приложение состоит из трех уровней, которые соединяются следующим образом:

1. Подключение базы данных

База данных PostgreSQL подключалась к приложению через **Entity Framework Core**. Это технология-посредник, которая позволяет работать с базой данных через объекты C#, не писать SQL-запросы вручную.

2. Соединение сервисов с базой данных

Классы, отвечающие за бизнес-логику (PartnerService), получают доступ к базе данных через **внедрение зависимостей** — ссылка на контекст базы данных передается в сервис через конструктор.

3. Соединение интерфейса с сервисами

Окна приложения (MainWindow, PartnerEditWindow) создают экземпляры сервисов и вызывают их методы для получения данных или их сохранения.

4. Соединение интерфейса с данными

Элементы интерфейса привязываются к данным через **механизм привязки данных (Data Binding)** WPF — при изменении данных интерфейс обновляется автоматически.

3.1.8. Верификация и аттестация программного обеспечения;

Процесс верификации (проверка соответствия требованиям):

1. **Проверка функциональных требований:**

Требование	Реализация
Просмотр списка партнеров	Карточки партнеров в левой колонке
Добавление партнера	Кнопка в меню "Партнеры" → "Добавить партнера"
Редактирование партнера	Кнопка редактировать под списком заказов
Удаление партнера	Кнопка удалить под списком заказов
История продаж	Таблица в правой колонке
Расчет скидки	Отображается на карточке партнера

2. **Проверка нефункциональных требований:**

Производительность - загрузка списка партнеров < 1 секунды

Надежность - обработка всех исключений

Удобство использования - интуитивно понятный интерфейс

Безопасность - валидация всех вводимых данных

Процесс аттестации (приемочное тестирование):

1. **Тестирование пользовательского интерфейса:**

   Проверка всех элементов управления

   Проверка навигации

   Проверка отображения данных

2. **Тестирование сценариев использования:**

Сценарий 1: Добавление нового партнера

- Открыть меню "Партнеры" → "Добавить партнера"
- Заполнить все поля формы
- Нажать "Сохранить"
- Новый партнер появился в списке

Сценарий 2: Просмотр истории продаж

- Кликнуть на карточку партнера
- В правой колонке отобразилась информация о партнере
- Загрузилась таблица с заказами
- Отобразилась общая сумма и скидка

Сценарий 3: Редактирование партнера

- Выбрать партнера
- Нажать кнопку редактировать
- Изменить данные
- Нажать "Сохранить"
- Данные обновились в карточке

Сценарий 4: Удаление партнера

- Выбрать партнера
- Нажать кнопку удалить
- Подтвердить удаление
- Партнер исчез из списка

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 по 17 марта 2025 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. **Разработано полнофункциональное приложение** "Учет партнеров" для компании «Мастер пол», реализующее:
   - Просмотр списка партнеров в виде карточек
   - Добавление и редактирование данных о партнерах
   - Просмотр истории реализации продукции
   - Автоматический расчет скидок на основе объема продаж
2. **Спроектирована и создана база данных** в PostgreSQL в 3 нормальной форме, включающая таблицы: partner_types, partners, products, sales_history.
3. **Разработаны модульные тесты** (23 теста), покрывающие все ключевые функции приложения, особенно расчет скидок для различных граничных значений.
4. **Применены современные технологии разработки:**
   - .NET 8.0 и C# 10.0
   - WPF для создания пользовательского интерфейса
   - Entity Framework Core для работы с БД
   - Git для контроля версий
5. **Обеспечено качество кода** через:
   - Соблюдение стандартов кодирования C#
   - Обработку исключительных ситуаций
   - Документирование кода XML-комментариями
6. **Проведено тестирование** всех сценариев использования приложения, подтвердившее его корректную работу.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. **ГОСТ Р ИСО/МЭК 12207-2010.** Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – М.: Стандартинформ, 2011. – 104 с.
2. **ГОСТ 19.101-77.** Единая система программной документации. Виды программ и программных документов. – М.: Стандартинформ, 2010. – 4 с.
3. **ГОСТ 19.701-90.** Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. – М.: Стандартинформ, 2010. – 26 с.
4. **Федорова, Г. Н.** Участие в интеграции программных модулей : учебное пособие / Г.Н. Федорова. – Москва : Академия, 2016. – 304 с. – ISBN 978-5-4468-1435-6.
5. **Гагарина, Л. Г.** Технология разработки программного обеспечения : учебное пособие / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. – Москва : ИНФРА-М, 2013. – 400 с. – ISBN 978-5-16-006959-7. – URL: https://znanium.com/catalog.php?bookinfo=389963 (дата обращения: 15.03.2026).
6. **Черников, Б. В.** Управление качеством программного обеспечения : учебник / Б.В. Черников. – Москва : ФОРУМ : ИНФРА-М, 2012. – 240 с. – ISBN 978-5-8199-0356-1. – URL: https://znanium.com/catalog.php?bookinfo=256901 (дата обращения: 15.03.2026).
7. **Рудаков, А. В.** Технология разработки программных продуктов : учеб. пособие для студ. сред. проф. образования / А.В. Рудаков. – Москва : Академия, 2010. – 208 с. – ISBN 978-5-7695-8156-4.